

Answer any three questions (including subparts) from Section A and Section B is compulsory

Section A

Answer any three questions (including subparts)

- 1 a) Let an array be represented as $A(2:20, 3:30, 1:25, 0:25, 5:30)$. Assuming row major representation of the array (using sequential byte addressing scheme) with 2 bytes per element. If k is the address of $A(4,5,6,7,5)$ then the address of the element represented as $A(3,10,15,6,12)$ is _____ [Fill in the blank correctly].

- b) If $f(x) = 2+6x+18x^2 + 54x^3 \dots + \infty$ and $f(x) = O(g(x))$ then $g(x) =$ _____ [Fill in the blank correctly].

(4+2)

- 2 a) For algorithms specified below, which of the following algorithm is faster [requires less time in terms of total number of steps (steps count)] when:

i) $m > n$

ii) $m < n$

[Explain your answer with suitable justification]

Algorithm Add1(a,b,c,m,n)

```
{  
  for (i=1; i<= m; i++)  
    for (j=1; j<= n; j++)  
      c[i,j] = a[i,j] + b[i,j];  
}
```

Algorithm Add2(a,b,c,m,n)

```
{  
  for (j=1; j<= n; j++)  
    for (i=1; i<= m; i++)  
      c[i,j] = a[i,j] + b[i,j];  
}
```

- b) Let a sparse matrix be stored in an array $X(0:t, 0:2)$ where t is the number of nonzero elements. The elements $X(0,0)$ and $X(0,1)$ contain the total the number of rows and the total number of columns of the matrix respectively. $X(0,2)$ contains the total number of nonzero elements of the matrix. Let $X(i,0)$, $X(i,1)$ and $X(i,2)$ contain row number, column number and the value of the nonzero element in the matrix respectively, for $1 \leq i \leq t$. Write an algorithm (using C pseudocode) to store two sparse matrices (taken as input from the user) in the form (as specified above using X), perform addition of these matrices, store the result in the form (as specified above using X) and display the result (in the form which is usually understood by an user unfamiliar with the representation specified using X). Specify the space and time complexity of your algorithm.

(2+4)

- 3 a) For the following algorithm (*tower*), derive its time complexity [Show all the steps of your derivation]. The recursion stack space used by this algorithm (*tower*) is at least 2^{n+1} [Fill in the blank correctly].

Algorithm *tower*(n , source, inter, dest)

```
{
    if( $n > 0$ )
    {
        tower( $n-1$ , source, dest, inter);  $T_R(n-1)$ 
        print("Move disk",  $n$ , "from", source, "to", dest);
        tower( $n-1$ , inter, source, dest);  $T_R(n-1)$ 
    }
}
```

$$n=0 \quad T_R=1$$

- b) If a polynomial is represented in the form $(m, e_{m-1}, b_{m-1}, e_{m-2}, b_{m-2}, \dots, e_0, b_0)$ where, the first entry m represents the number of non-zero terms followed by two entries (e_i, b_i) for each non-zero term (representing an exponent-coefficient pair) in the order $e_{m-1} > e_{m-2} > \dots > e_0 \geq 0$. The total number of element locations needed to store the result using the above representation when the polynomial $x^{1000} + 4x^{500} + 3x^5 + 2x^4 + 5x + 2$ is added to the polynomial $x^{500} + 4x^{200} + 9x^4 + 2x^3 + 10x + 1$ is 1000 [Fill in the blank correctly].

(4+2)

- 4 a) Consider the following algorithm

Algorithm *fib_t*(n , curr, prev)

```
{
    if ( $n == 1$ )
        return prev;
    return fib_t( $n - 1$ , curr + prev, curr);
}
```

If each formal parameter, local variable (if any) and return address require one word of storage then the recursion stack space needed by the compiler (which is said to implement tail recursion) for the implementation of the following algorithm is at least _____. [Select the correct option from the following]

- i) n
- ii) c , where c is a constant

- ii) $n+1$
- iv) $n-1$

b) Consider the following algorithm

```

Algorithm polyadd (a, b, c)
{
    p=1; q=1; r=1;
    m=a[0]; n=b[0];
    while(p<=2*m && q<=2*n)
    {
        if(a[p] == b[q])
        {
            c[r+1]=a[p+1]+b[q+1];
            if(c[r+1]!=0)
            {
                c[r]=a[p]; r=r+2;
            }
            p=p+2; q=q+2;
        }
        if(a[p]>b[q])
        {
            c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
        }
        if(a[p] < b[q])
        {
            c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
        }
    }
    while(p<=2*m)
    {
        c[r+1]=a[p+1]; c[r]=a[p]; p=p+2; r=r+2;
    }
    while(q<=2*n)
    {
        c[r+1]=b[q+1]; c[r]=b[q]; q=q+2; r=r+2;
    }
    c[0]=(r-1)/2;
}
    
```

The worst case time complexity of the algorithm *polyadd* is _____ [Select the correct option from the following]

- a) $O(m+n)$
- b) $O(m)$
- c) $O(n)$
- d) $O(mn)$

(2+4)

Section B

Compulsory

5. Write down algorithms (C pseudocode) to perform the following operations on a circular queue of size n (implemented using array) depending on users choice (where the value of n is provided by the user as input) and specify their time complexities using asymptotic notations.
- i) Insert k number of elements (provided by the user as input) to the circular queue (where the value of k is also taken from the user as input) *enqueue*
 - ii) Delete an element from the circular queue and display the deleted element *dequeue*
 - iii) Display the contents of the circular queue *display*
- (6)
6. Write down algorithms (C pseudocode) to perform the following operations on a stack of size n (implemented using array) depending on users choice (where the value of n is provided by the user as input) and specify their time complexities using asymptotic notations

- i) Push an element (provided by the user as input) to the stack
 - ii) Pop k number of elements from the stack (where the value of k is taken from the user as input) and print the popped (deleted) elements
 - iii) Display the contents of the stack
- (6)
